

Secure API Testing & Authorization Validation

Objective:

The objective of this task is to evaluate the security posture of REST API endpoints by testing authentication mechanisms, authorization controls, input validation, and rate limiting. The assessment was performed to identify common API misconfigurations and vulnerabilities based on the OWASP API Security Top 10 risks.

Tools Used:

- Tool: Postman
- Environment: Kali Linux
- Target: JSON Placeholder (Public REST API for testing)

Target API Information:

- Base URL: <https://jsonplaceholder.typicode.com>
- Tested Endpoints: /users, /users/{id}
- HTTP Methods Used: GET

Methodology:

The API was tested using manual requests in Postman. Various scenarios were executed, including:

- Requests with and without authentication headers
- Accessing different resource IDs
- Sending malformed inputs
- Rapid repeated requests
- Observing HTTP responses and error handling

Test Cases and Results:

Test Case 1 – Basic API Access

Request:

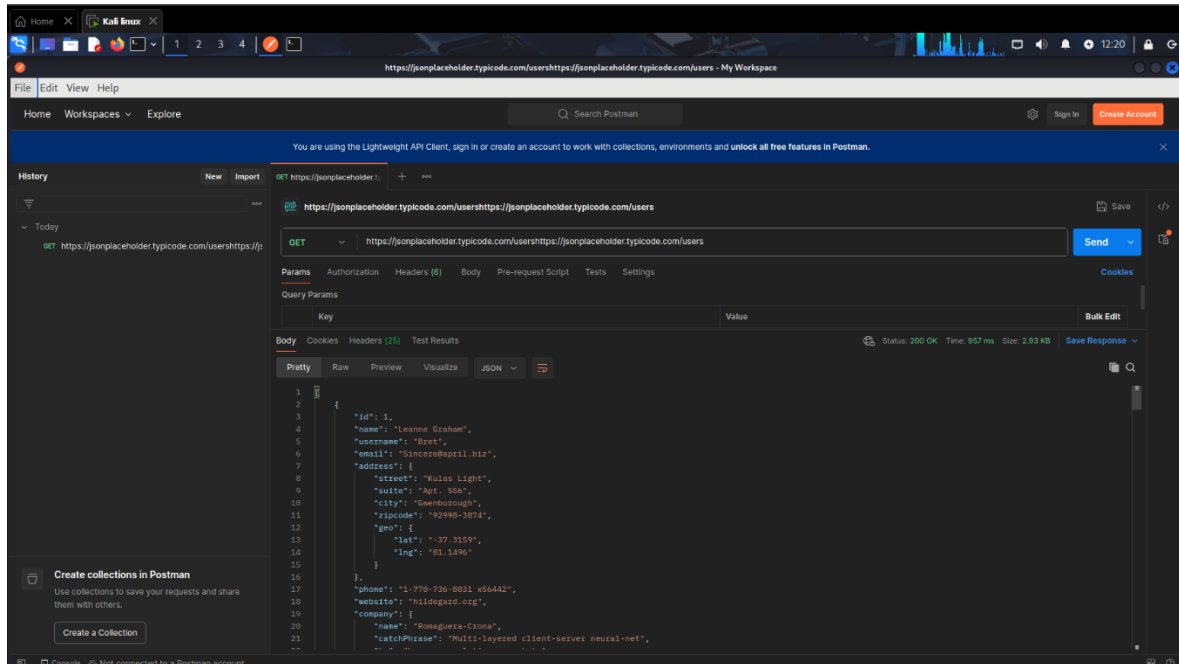
GET /users

Observation:

The API returned a list of users successfully with status code 200 OK.

Result:

The endpoint is publicly accessible.



Test Case 2 – Authentication Header Validation

Request:

GET /users

Authorization:

Bearer testtoken123

Observation:

The request was accepted and returned 200 OK even with an invalid token.

Result:

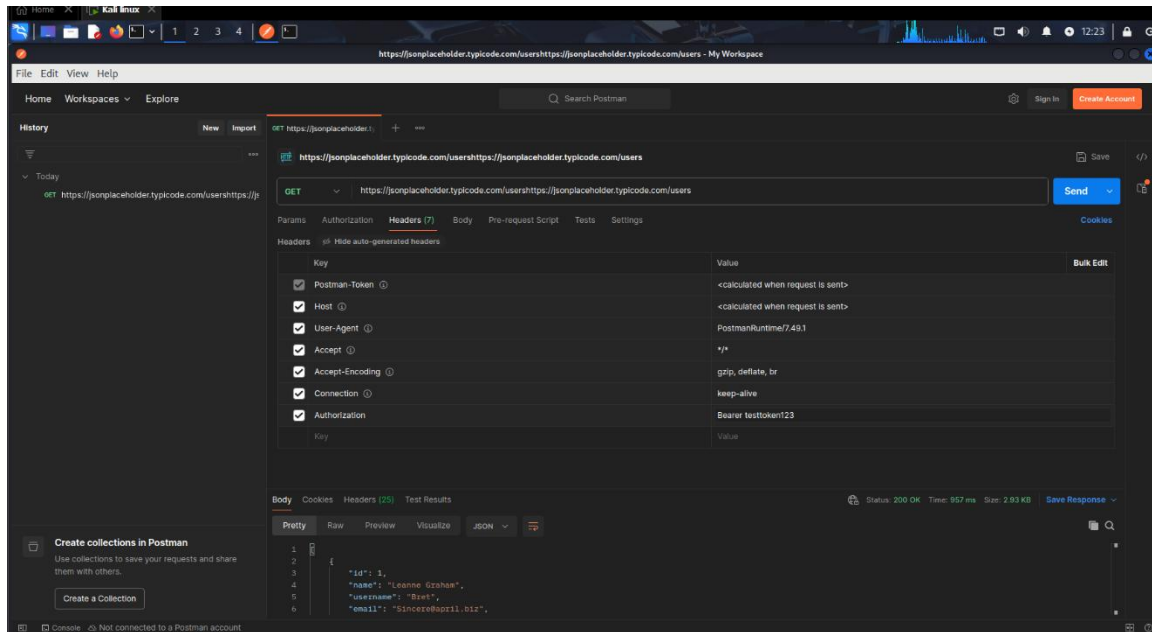
Authentication header was not validated.

Risk:

Improper authentication enforcement.

OWASP Mapping:

API2 – Broken Authentication



Test Case 3 – Unauthenticated Access

Request:

GET /users (no Authorization header)

Observation:

The API still returned 200 OK and exposed all user data.

Result:

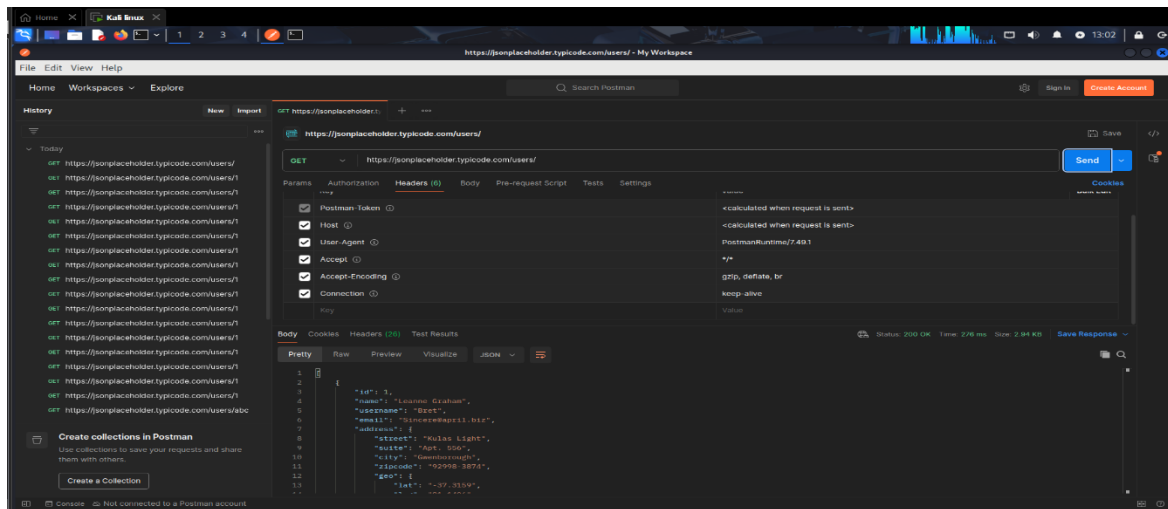
Access to sensitive data without authentication.

Risk:

Unauthorized data exposure.

OWASP Mapping:

API2 – Broken Authentication



Test Case 4 – Broken Authorization (IDOR)

Requests:

- GET /users/1
- GET /users/2
- GET /users/3

Observation:

Different user records were accessible by simply modifying the user ID in the URL without any login or authorization.

Result:

Direct access to other users' data.

Risk:

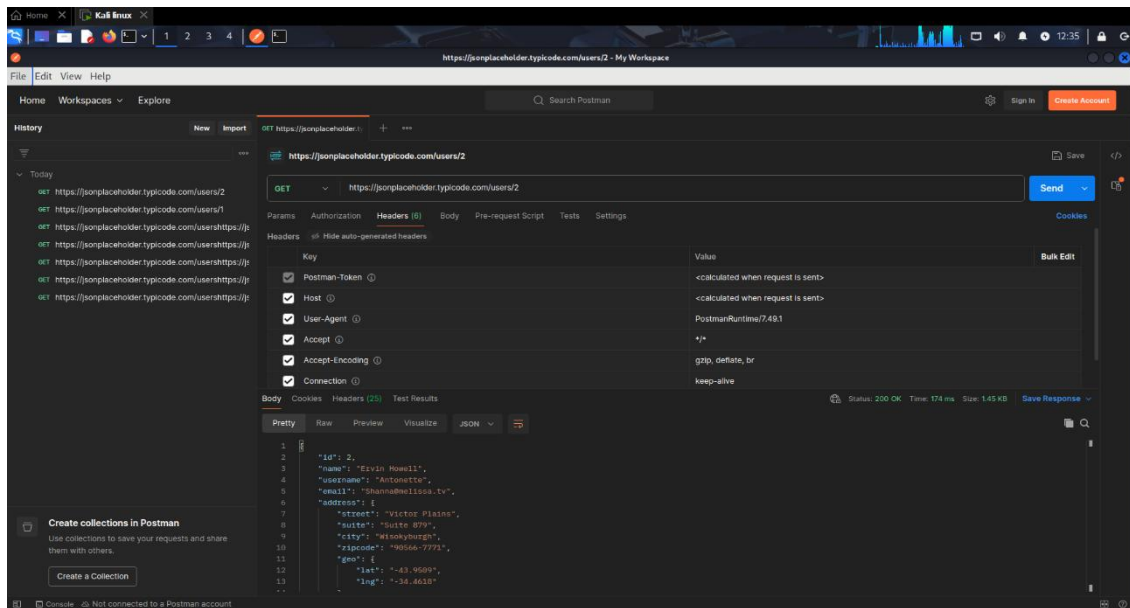
Broken Object Level Authorization (IDOR).

Impact:

Attackers could enumerate and access other users' sensitive information.

OWASP Mapping:

API1 – Broken Object Level Authorization (BOLA/IDOR)



Test Case 5 – Input Validation Testing

Requests:

- GET /users/abc
- GET /users/-1
- GET /users/9999

Observation:

The API handled invalid inputs with proper 404 Not Found responses and did not reveal internal server details.

Result:

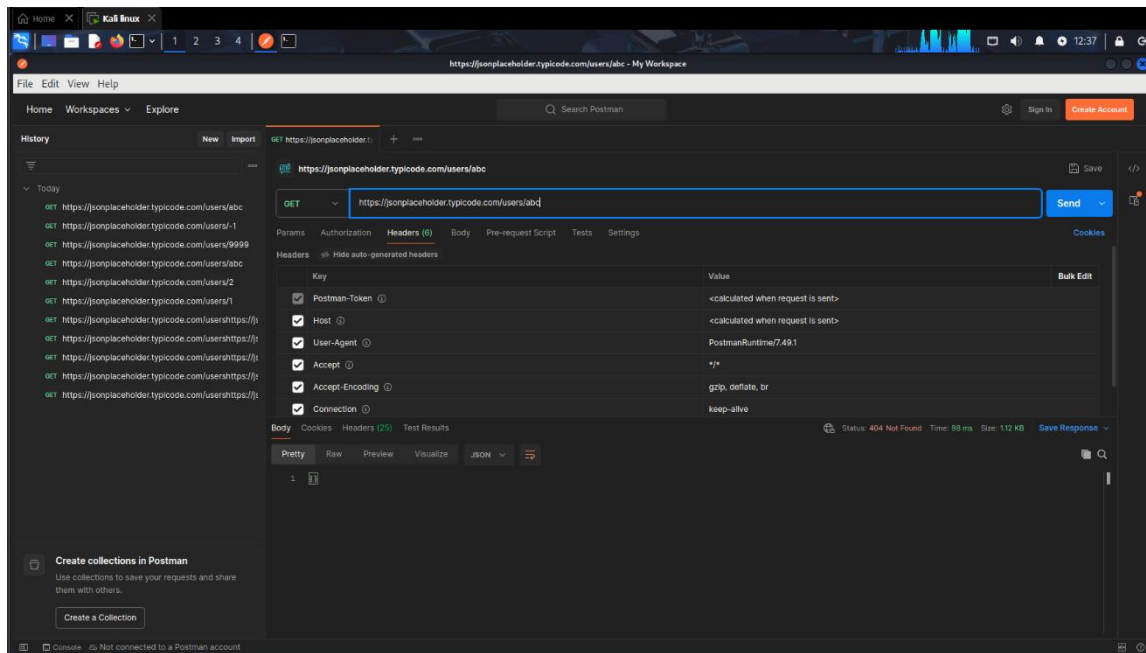
Basic input validation is present.

Risk:

Low

OWASP Mapping:

API8 – Injection / Input Validation



Test Case 6 – Rate Limiting Check

Action:

Multiple rapid requests (15–20) were sent consecutively.

Observation:

All requests were processed successfully without blocking or throttling.

Result:

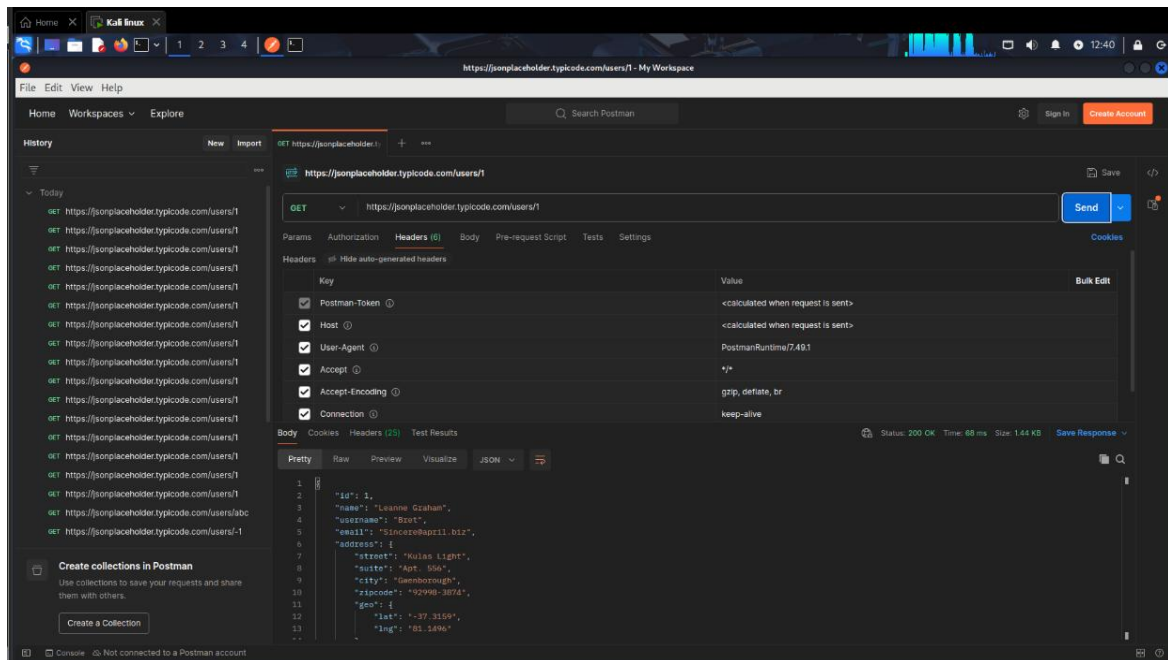
No rate limiting detected.

Risk:

Possible abuse through brute force or automated attacks.

OWASP Mapping:

API4 – Lack of Resources & Rate Limiting



Identified Security Issues:

Issue	Risk Level	OWASP Category
No authentication enforcement	Medium	API2
Access to other users' data (IDOR)	High	API1
No rate limiting	Medium	API4

Recommendations:

The following security improvements are recommended:

- Implement token-based authentication (JWT or OAuth)
- Enforce strict authorization checks for each resource
- Validate user identity before returning data
- Add rate limiting and request throttling
- Log suspicious or repeated requests
- Apply proper access control mechanisms

Conclusion:

The API security testing exercise demonstrated how improper authentication and authorization controls can lead to unauthorized access and data exposure. The assessment highlights the importance of enforcing authentication, validating user permissions, and implementing rate limiting to secure REST APIs.

Through this task, practical experience was gained in:

- Using Postman for API testing
- Analyzing HTTP requests and responses
- Identifying authorization flaws
- Mapping vulnerabilities to OWASP API Security risks